Sprzęt: konwerter USB/RS485, moduł RS485-TTL (MAX485), Arduino UNO, analizator stanów logicznych, moduł Modbus Slave wskazany przez prowadzącego, rezystory 120 Ω, multimetr, przewody.
Oprogramowanie: terminal portu szeregowego (dowolny), QModMaster, Arduino IDE, oprogramowanie analizatora stanów logicznych, Python 3 z pakietami pyserial i pymodbus.

Organizacja: instrukcja realizowana na dwóch zajęciach (2 × 90 min). **Zajęcia 1:** przygotowanie magistrali, zadanie 1 (etapy A i B), zadanie 2 — punkty 1–4. **Zajęcia 2:** zadanie 2 — punkty 5–8, zadanie 3 — etap A. **Praca własna:** etap B zadania 3; działanie programu przedstawić prowadzącemu, wyniki opisać w sprawozdaniu. Szkic urządzenia slave do etapu A warto przygotować przed drugimi zajęciami.

Przygotowanie magistrali RS-485

1. Połączyć urządzenia w topologii liniowej: A z A, B z B. Unikać odgałęzień dłuższych niż kilkadziesiąt centymetrów.
2. Przy wyłączonym zasilaniu zmierzyć multimetrem rezystancję między liniami A i B. Poprawna terminacja (dwa rezystory 120 Ω na końcach magistrali) daje ok. **60 Ω**. Wynik zapisać.
3. Jeżeli pomiar wynosi ok. 120 Ω (brak jednego terminatora) lub wyraźnie mniej niż 60 Ω (terminatory włączone w kilku urządzeniach) — skorygować terminację zworkami lub rezystorami.

Uwaga: producenci różnie oznaczają linie RS-485 (A/B, D+/D−, Data+/Data−). Jeżeli mimo poprawnych ustawień nie ma komunikacji, pierwszym krokiem jest zamiana przewodów A i B.

Zadanie 1: Miniprotokół tekstowy — od UART do RS-485

Cel: konfiguracja łącza szeregowego i formatu znaku, test prostego protokołu tekstowego, a następnie przeniesienie go na magistralę RS-485 ze sterowaniem kierunkiem nadawania.

Etap A — komunikacja przez wbudowany konwerter USB Arduino

Sprzęt: PC, Arduino UNO, przewód USB.
Połączenia: Arduino podłączone do komputera wyłącznie przewodem USB. Wbudowany konwerter USB–UART płytki łączy się z mikrokontrolerem przez piny 0 (RX) i 1 (TX), więc komputer „widzi" Arduino jako port COM.
Do wykonania:

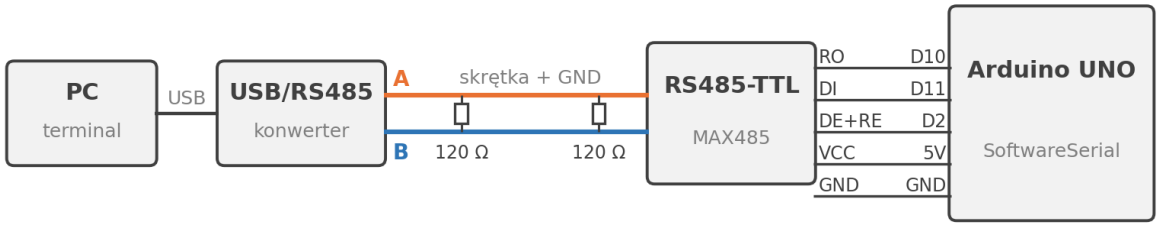
1. Wgrać do Arduino szkic „echo-komend":
   - przyjmuje linie ASCII zakończone \n,
   - komendy: PING → PONG, LED ON|OFF, ANALOG? → zwraca A0=xxx.
2. W terminalu wybrać port COM Arduino i ustawić: **115200 b/s, 8N1, bez kontroli przepływu**, zakończenie linii **LF** (lub CR+LF).
3. Wysłać komendy PING, LED ON, LED OFF, ANALOG? i zapisać log z czasem.
4. Weryfikacja: poprawna odpowiedź na każdą komendę, brak „krzaków". Następnie celowo ustawić w terminalu 9600 b/s i opisać efekt.
5. Rozszerzyć program o co najmniej trzy własne komendy (w tym jedną z parametrem, np. „LED BLINK 5"). Nieznane komendy i zbyt długie linie (np. ponad 32 znaki) mają zwracać odpowiedź ERR.

```
/*                    ECHO KOMEND                    */
void setup() {
  Serial.begin(115200);
  pinMode(LED_BUILTIN, OUTPUT);
}
void loop() {
  static String line;
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') {
      line.trim();
      if (line == "PING") Serial.println("PONG");
      else if (line == "LED ON") {
        digitalWrite(LED_BUILTIN, 1); Serial.println("OK");
      }
      else if (line == "LED OFF") {
        digitalWrite(LED_BUILTIN, 0); Serial.println("OK");
      }
      else if (line == "ANALOG?") {
        Serial.print("A0="); Serial.println(analogRead(A0));
      }
      else Serial.println("ERR");
      line = "";
    } else line += c;
  }
}
```

Etap B — ten sam protokół przez magistralę RS-485

Sprzęt: PC + konwerter USB/RS485, moduł RS485-TTL, Arduino UNO + przewód USB, przewody.

Połączenia: zgodnie z rysunkiem 1. Wejścia DE i RE modułu RS485-TTL należy zewrzeć i podłączyć do jednego pinu Arduino. Terminal łączy się teraz z portem COM konwertera USB/RS485, a przewód USB Arduino służy tylko do zasilania, wgrywania programu i komunikatów diagnostycznych.



Terminacja 120 Ω na obu końcach magistrali (w module lub konwerterze — sprawdź zworki)

Rys. 1. Schemat połączeń do zadania 1, etap B

Moduł RS485-TTL	Arduino UNO	Funkcja
RO	D10	dane odebrane z magistrali (RX SoftwareSerial)
DI	D11	dane do nadania (TX SoftwareSerial)
DE + RE (zwarte)	D2	kierunek: 1 – nadawanie, 0 – odbiór
VCC / GND	5V / GND	zasilanie modułu

Dlaczego inne piny? Sprzętowy UART Arduino UNO (piny 0 i 1) jest już zajęty przez wbudowany konwerter USB z etapu A. Komunikację RS-485 realizuje więc SoftwareSerial na pinach 10 i 11. Pracuje on niezawodnie przy prędkościach do ok. 38 400 b/s — dlatego w etapie B używamy 19 200 b/s. Nowym elementem jest sterowanie kierunkiem: magistrala RS-485 jest półdupleksowa, więc nadajnik modułu włącza się tylko na czas wysyłania odpowiedzi.

Do wykonania:

6. Wgrać zmodyfikowany szkic (poniżej). Porównać go z wersją z etapu A i wskazać różnice.

```
#include <SoftwareSerial.h>

const byte PIN_RX  = 10;   // RO modułu RS485-TTL
const byte PIN_TX  = 11;   // DI modułu RS485-TTL
const byte PIN_DIR = 2;    // DE + RE (zwarte)
SoftwareSerial rs485(PIN_RX, PIN_TX);

void sendLine(const String& s) {
  digitalWrite(PIN_DIR, HIGH);   // nadawanie
  rs485.print(s);
  rs485.print("\r\n");
  digitalWrite(PIN_DIR, LOW);    // odbiór
}  // write() w SoftwareSerial kończy się po wysłaniu bitu stopu

void setup() {
  pinMode(PIN_DIR, OUTPUT);
  digitalWrite(PIN_DIR, LOW);
  pinMode(LED_BUILTIN, OUTPUT);
  rs485.begin(19200);
  Serial.begin(115200);          // tylko diagnostyka przez USB
}

void loop() {
  static String line;
  while (rs485.available()) {
    char c = rs485.read();
    if (c == '\n') {
      line.trim();
      Serial.println("RX: " + line);
      if (line == "PING") sendLine("PONG");
      else if (line == "LED ON") {
        digitalWrite(LED_BUILTIN, HIGH); sendLine("OK");
      }
      else if (line == "LED OFF") {
        digitalWrite(LED_BUILTIN, LOW); sendLine("OK");
      }
      else if (line == "ANALOG?") sendLine("A0=" + String(analogRead(A0)));
      else sendLine("ERR");
      line = "";
    } else if (line.length() < 32) {
      line += c;
    }
  }
}
```

7. W terminalu wybrać port COM **konwertera USB/RS485** i ustawić 19 200 b/s, 8N1, zakończenie linii LF. Powtórzyć test komend z etapu A. Przenieść do tej wersji własne komendy z punktu 5.
8. Podłączyć analizator do linii RO i DI modułu RS485-TTL (dekoder UART: 19 200 b/s, 8N1). Zarejestrować komendę PING i odpowiedź PONG. Zmierzyć czas trwania jednego bitu i jednego znaku, sprawdzić kolejność bitów (LSB pierwszy) dla znaku „P" (0x50).
9. Dodatkowym kanałem analizatora zarejestrować sygnał kierunku (D2). Opisać, kiedy nadajnik jest włączany i wyłączany względem ramki odpowiedzi.
10. Test: w funkcji sendLine() przenieść instrukcję digitalWrite(PIN_DIR, LOW) przed wysłanie tekstu (lub usunąć sterowanie kierunkiem) i opisać, co zobaczy terminal i analizator.

Sprawozdanie powinno zawierać: log z terminala z etapu A i B, schemat połączeń etapu B, zrzut z analizatora z opisem bitów jednego znaku oraz zmierzonym czasem bitu, przebieg sygnału DE/RE względem odpowiedzi, wynik testu z punktu 10, opis własnego miniprotokołu (tabela komend i odpowiedzi).

Zadanie 2: Modbus RTU z dedykowanym modułem (wskazanym przez prowadzącego)

Cel: praktyka z protokołem Modbus RTU (ramki, funkcje, adresacja, kody wyjątków, CRC).
Sprzęt: PC + USB/RS485, dedykowany moduł Modbus RTU (np. moduł I/O), przewody, rezystory 120 Ω, RS485-TTL, analizator stanów logicznych.
Do wykonania (PC jako master):

1. Na podstawie dokumentacji modułu uzupełnić tabelę parametrów komunikacji i rejestrów wykorzystanych w zadaniu.

Parametr	Wartość (z dokumentacji modułu)
Adres urządzenia (Slave ID)	
Prędkość / format znaku	..... b/s, ... bitów danych, parzystość ....., bity stopu .....
Rejestr 1: numer, funkcja, znaczenie, skalowanie	
Rejestr 2: numer, funkcja, znaczenie, skalowanie	
Rejestr do zapisu (jeśli istnieje)	

2. Skonfigurować narzędzie master (QModMaster lub modpoll): port, prędkość, parzystość, tryb RTU, Slave ID. Sprawdzić w ustawieniach programu adres bazowy (Base Addr) i zanotować, czy adres wpisywany w oknie jest adresem przesyłanym w ramce.
3. Odczytać wybrane dane zgodnie z dokumentacją. Zanotować odpowiedzi i przeliczyć je na wartości fizyczne (np. U = kod / 4096 · 5 V).
4. Zapisać wybrane dane do urządzenia (funkcja 06 lub 16, jeśli urządzenie na to pozwala) i potwierdzić zapis ponownym odczytem.
5. Wywołać **odpowiedź wyjątku**: odczytać rejestr spoza mapy urządzenia. Zapisać ramkę odpowiedzi i kod wyjątku.
6. Ustawić **błędny Slave ID** i wysłać zapytanie. Opisać zachowanie (brak odpowiedzi — timeout) i wyjaśnić, dlaczego nie jest to odpowiedź wyjątku.

7. Zarejestrować analizatorem pełną transakcję (zapytanie i odpowiedź) dla odczytu i dla zapisu. Zmierzyć czas trwania ramki zapytania oraz czas od końca zapytania do początku odpowiedzi (czas reakcji urządzenia).
8. Obliczyć CRC-16 zarejestrowanego zapytania poniższym programem i porównać wynik z dwoma ostatnimi bajtami ramki.

```
def crc16_modbus(data):
    crc = 0xFFFF
    for b in data:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc


ramka = [0x01, 0x03, 0x00, 0x00, 0x00, 0x02]   # wpisz bajty zapytania
c = crc16_modbus(ramka)
print(f'CRC: {c & 0xFF:02X} {c >> 8:02X}')   # młodszy bajt pierwszy: C4 0B
```

Sprawozdanie powinno zawierać: tabelę parametrów, schemat połączeń, zrzuty ekranu odczytu i zapisu, zarejestrowane ramki z interpretacją bajt po bajcie, porównanie odpowiedzi wyjątku z timeoutem, obliczenie CRC oraz zmierzone czasy.

Zadanie 3: PC ↔ Arduino po Modbus RTU (własne urządzenie slave i własny master w Pythonie)

Cel: zaprojektować i uruchomić kompletny tor komunikacji Modbus RTU pomiędzy PC (master) a Arduino (slave): stworzyć urządzenie slave na Arduino (mapa rejestrów + logika), a następnie napisać własnego klienta w Pythonie, który odczytuje i zapisuje dane.
Sprzęt: PC + USB/RS485, Arduino UNO, moduł RS485-TTL (MAX485), przewody, rezystory 120 Ω (terminacja na końcach magistrali).
Połączenia: takie same jak w zadaniu 1, etap B (rysunek 1): RO → D10, DI → D11, DE + RE → D2, parametry transmisji 19 200 b/s, 8N1.

> **Uwaga:** do obsługi Modbus RTU użyć biblioteki wskazanej przez prowadzącego, która pozwala pracować na porcie SoftwareSerial i sterować pinem kierunku (DE/RE). Sprzętowy UART (piny 0 i 1) pozostaje wolny — przez USB można wgrywać program i wysyłać komunikaty diagnostyczne bez odłączania magistrali. Ponieważ SoftwareSerial wysyła i odbiera znak po znaku, nie należy w pętli programu używać długich opóźnień (delay), bo urządzenie przestanie odpowiadać w wymaganym czasie.

Etap A — urządzenie slave z zadaną mapą rejestrów

Zaimplementować na Arduino poniższą mapę rejestrów i sprawdzić ją w QModMaster:

Obszar	Adres	Dostęp	Znaczenie
Coil	0	R/W	dioda LED (1 – świeci)
Discrete Input	0	R	alarm: pomiar A0 > próg
Input Register	0	R	pomiar z wejścia A0 (0–1023)
Input Register	1	R	czas pracy urządzenia [s]
Holding Register	0	R/W	próg alarmu (0–1023), wartość domyślna 512
Holding Register	1	R/W	liczba próbek filtru średniej (1–16)

Etap B — własna mapa, logika i klient w Pythonie (praca własna)

1. Zaprojektować własną mapę rejestrów: tabela (typ: Coil/Discrete/Input/Holding; adres; R/W; jednostka; zakres; opis logiki).
2. Zaimplementować Arduino (slave): odwzorować zaprojektowaną mapę; uaktualniać rejestry; zaimplementować logikę (alarmy z histerezą, liczniki, filtry); zapewnić poprawne odpowiedzi Modbus, w tym wyjątki dla nieistniejących adresów.
3. Napisać aplikację w Pythonie (master):
   - zrealizować odczyt i zapis wszystkich elementów mapy;
   - przygotować minimum dwa scenariusze testowe (np. kalibracja + odczyt; próg + alarm);
   - obsłużyć błędy komunikacji (timeout, odpowiedź wyjątku).
4. Przeprowadzić testy funkcjonalne: wykazać poprawne sterowanie i konfigurację z poziomu Pythona oraz zgodność wartości wyliczanych z założeniami.

Szkielet klienta (pymodbus 3.x):

```
from pymodbus.client import ModbusSerialClient


client = ModbusSerialClient(port='COM5', baudrate=19200, bytesize=8,
                            parity='N', stopbits=1, timeout=1)
client.connect()
# w nowszych wersjach pymodbus zamiast slave=1 jest device_id=1
rr = client.read_input_registers(0, count=2, slave=1)
if rr.isError():
    print('Błąd:', rr)
else:
    print('A0 =', rr.registers[0], ' czas pracy =', rr.registers[1], 's')
client.write_register(0, 600, slave=1)   # nowy próg alarmu
client.close()
```

Sprawozdanie powinno zawierać: tabele rejestrów z opisem i parametry komunikacji, kod programu slave i mastera (najważniejsze fragmenty), wyniki testów funkcjonalnych — zrzuty ekranów z opisem.

Pytania kontrolne

1. Ile trwa przesłanie jednego znaku w formacie 8N1 przy 19 200 b/s? Porównaj wynik z pomiarem z zadania 1 (etap B).
2. Dlaczego błędny Slave ID nie powoduje odpowiedzi wyjątku?
3. Co zobaczysz na analizatorze, jeśli sygnał DE zostanie wyłączony przed wysłaniem ostatniego bajtu?
4. Jaką rezystancję zmierzysz między A i B, jeśli terminator 120 Ω będzie włączony w trzech urządzeniach?